

Reviewer Pack — Minimal Representation Rank of Quantum States and DPLL Proof...

Entry 95e73bab9487 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 18:21:26 UTC

Minimal Representation Rank of Quantum States and DPLL Proof Length

Entry ID: 95e73bab9487 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 18:21:26 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Representation Theory) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every quantum state described by a Hilbert space of dimension n , the minimal representation rank of its matrix representation is linearly correlated with the length of the shortest DPLL proof for a given CNF instance, such that $\min_rank(M) = \Theta(L)$, where L is the length of the shortest DPLL proof and M is the matrix representation of the quantum state.

Rationale (proposer's reasoning):

Quantum information theory's representation theory provides a framework to encode classical information in quantum states. Minimal representation rank could potentially expose hidden structures in the complexity of Boolean satisfiability problems, offering new insights into proof length and complexity classes.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fad736324e8d9003

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all quantum states of dimension $n \leq 40$ and corresponding CNF instances, the minimal representation rank of the matrix representation ($\min_rank(M)$) correlates with the length of the shortest DPLL proof (L) across 30 random seeds. Specifically, $\min_rank(M) = \Theta(L)$ must be true for at least 80% of the correlations (support_fraction ≥ 0.8) and the mean absolute difference between $\min_rank(M)$ and L over all seeds must be less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [1 if i == j else 0 for j in range(n)] for i, row in enumerate(matrix)]

    for i in range(n):
        # Find pivot
        max_row = i
        for r in range(i+1, n):
            if abs(augmented_matrix[r][i]) > abs(augmented_matrix[max_row][i]):
                max_row = r

        # Swap rows
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

        # Eliminate
        for r in range(i+1, n):
            factor = Fraction(augmented_matrix[r][i], augmented_matrix[i][i])
            for c in range(n + 1):
                augmented_matrix[r][c] -= factor * augmented_matrix[i][c]

    # Back substitution
    min_rank = n
    for r in range(n-1, -1, -1):
        if all(augmented_matrix[r][c] == 0 for c in range(r+1, n)):
            min_rank -= 1

    return min_rank

def random_quantum_state(n):
```

```

state = [[random.random() for _ in range(n)] for _ in range(n)]
# Normalize the state
norm = sum(a**2 + b**2 for row in state for a, b in zip(row[:n//2], row[n//2:]))
for i in range(n):
    state[i] = [x / math.sqrt(norm) for x in state[i]]
return state

def dpll_solver(cnf):
    def solve(assignment, clauses):
        if not clauses:
            return True
        clause = next(clause for clause in clauses if any(lit in assignment and assignment[lit] ==
pos_lit = next(lit for lit in clause if lit > 0)
neg_lit = -pos_lit
if pos_lit not in assignment:
    assignment[pos_lit] = 1
    if solve(assignment, clauses):
        return True
    del assignment[pos_lit]
if neg_lit not in assignment:
    assignment[neg_lit] = 1
    if solve(assignment, clauses):
        return True
    del assignment[neg_lit]
    return False

    n = len(cnf)
    assignment = {}
    return solve(assignment, cnf)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        quantum_state = random_quantum_state(n)
        matrix_representation = [[sum(a * b for a, b in zip(row1, row2)) for row2 in quantum_state

        # Convert to CNF instance (simplified example)
        cnf = []
        for i in range(n):
            for j in range(i+1, n):
                cnf.append([i+1, -(j+1)])

        dpll_length = dpll_solver(cnf)

        if dpll_length is None:
            return {
                "metric_name": "DPLL Proof Length",
                "metric_value": None,
                "instances_tested": 0,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "DPLL solver failed"

```

```

    }

    min_rank = gaussian_elimination(matrix_representation)

    results.append({
        "n": n,
        "min_rank": min_rank,
        "dpll_length": dpll_length
    })

mean_diff = sum(abs(res["min_rank"] - res["dpll_length"]) for res in results) / len(results)
support_fraction = sum(1 for res in results if abs(res["min_rank"] - res["dpll_length"]) <= 3)

return {
    "metric_name": "DPLL Proof Length",
    "metric_value": mean_diff,
    "instances_tested": len(results),
    "n_max": max(res["n"] for res in results),
    "conjecture_holds": support_fraction >= 0.8,
    "counterexample": "" if support_fraction >= 0.8 else f"Difference > 3 at n={max(res['n'] for res in results)}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_diff = sum(res["metric_value"] for res in results if res["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_diff} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_diff} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Difference > 3\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1425f49.py", line 132, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^

```


2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/95e73bab9487.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/95e73bab9487.tar.gz` (if generated)